

AI-Assisted Software Development

Module 16 — Business AI in Practice · Notebook 51

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

2026-07-22

AI-Assisted Software Development

AI-Assisted Software Development
Module 16 — Business AI in Practice · Notebook 51

Prof. Dr. Christoph Weisser
HSBI
Summer Semester 2026

Contents

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

2026-07-22

AI-Assisted Software Development

└ Contents

Contents
1 IDEs
2 Git
3 Prompting
4 Review
5 When not

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

2026-07-22

AI-Assisted Software Development

└─ IDEs

└─ Outline

Outline

1 IDEs

2 Git

3 Prompting

4 Review

5 When not

The modern IDE landscape (2026)

Tool	AI assistance	Best for
VS Code + Copilot	inline completion + chat	broad default; Jupyter support
Cursor	chat-first; multi-file refactors	agentic edits, big rewrites
PyCharm + JetBrains AI	completion + assistant panel	large, refactor-heavy projects

Recommendation for beginners

VS Code + Copilot or **Cursor** — the most forgiving combinations to learn the workflow on.

2026-07-22

AI-Assisted Software Development

└ IDEs

└ The modern IDE landscape (2026)

~6 min. Point: orientation only — the *workflow* is the lecture, not the tool.

This slide dates fastest in the whole course. Say so out loud; it teaches them to hold tool choices loosely. If the table has drifted by the time you teach it, that is itself the lesson.

Steer beginners to VS Code + Copilot: Jupyter support matters for this course, and the inline model is easier to supervise than an agent that edits five files at once.

Ask the room: “Who’s already using one of these daily?” Calibrates how fast you can go, and surfaces students who can help their neighbours.

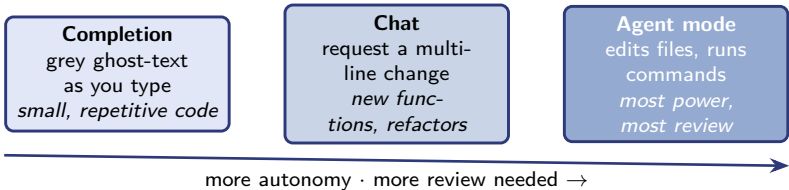
Don’t get drawn into a tool war — redirect to the review discipline, which is the transferable part.

Tool	AI assistance	Best for
VS Code + Copilot	inline completion + chat	broad default; Jupyter support
Cursor	chat-first; multi-file refactors	agentic edits, big rewrites
PyCharm + JetBrains AI	completion + assistant panel	large, refactor-heavy projects

Recommendation for beginners

VS Code + Copilot or **Cursor** — the most forgiving combinations to learn the workflow on.

Three ways the assistant helps — rising autonomy



The more the assistant does on its own, the more your review discipline matters (Module 10 — agents, tools & MCP — builds on Agent mode).

2026-07-22

AI-Assisted Software Development

└ IDEs

└ Three ways the assistant helps — rising autonomy



~8 min. Point: three modes, rising autonomy — and **review burden rises with autonomy**. That’s the whole slide.

Match mode to task: completion for boilerplate you could type; chat for a function you can describe; agent mode when the change genuinely spans files — and only then.

The trap with agent mode is that it produces so much so fast that review is skipped exactly when it matters most. Reviewing 400 generated lines is harder than writing 40 yourself.

Ask the room: “It edited eight files and says it’s done. What do you do first?” Read the diff — `git diff` before you run anything. That habit is the bridge to the review section.

Forward-reference Module 10: agent mode is the same architecture they’ll build.

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

2026-07-22

AI-Assisted Software Development

- └ Git
 - └ Outline

Outline

- IDEs
- Git**
- Prompting
- Review
- When not

Git — nine commands cover 90 % of daily work

Command	What it does
status / diff	see what changed
add / commit	stage and snapshot
pull / push	sync with the remote
checkout -b / checkout	make / switch a branch
merge	combine a branch back in

Common beginner mistake

Force-pushing over the main branch destroys history others built on. Force-push only your *own* feature branches.

2026-07-22

AI-Assisted Software Development

└─Git

└─Git — nine commands cover 90 % of daily work

Command	What it does
status / diff	see what changed
add / commit	stage and snapshot
pull / push	sync with the remote
checkout -b / checkout	make / switch a branch
merge	combine a branch back in

Common beginner mistake
Force-pushing over the main branch destroys history others built on. Force-push only your own feature branches.

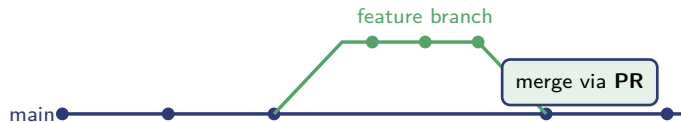
~10 min. Point: nine commands, and they are enough. Students overestimate how much Git they need and freeze.

Frame Git as the *safety net that makes AI assistance usable*: you can accept a big generated change precisely because you can throw it away. Committing before you prompt is the habit to instil — a clean working tree turns any bad generation into one git checkout.

The force-push warning is the one that saves someone’s week. Explain *why*: rewriting shared history means everyone else’s clone disagrees about what happened, and recovery is genuinely painful.

Ask the room: “The assistant rewrote a working file and made it worse. What’s the recovery?”
If they can’t answer, that’s why we commit first.

Branches and pull requests — the unit of professional work



A good PR

short title · a 2-paragraph *what / why / tested* · **small scope** (30 lines reviewed in 5 min) · tests or a screenshot.

AI-generated PRs are no exception — arguably *more* important to keep small.

2026-07-22

AI-Assisted Software Development

└ Git

└ Branches and pull requests — the unit of professional work



A good PR

short title · a 2-paragraph *what / why / tested* · **small scope** (30 lines reviewed in 5 min) · tests or a screenshot.

AI-generated PRs are no exception — arguably more important to keep small.

~**10 min.** Point: the PR is the unit of professional work — and the place review actually happens.

Walk the diagram once: branch off main, commit a few times, open a PR, merge back. The value is that main stays working while you experiment.

Small scope is the bullet to defend. A 30-line PR gets a real review in five minutes; a 900-line PR gets “LGTM” and a rubber stamp. Reviewer attention is the scarce resource, and it does not scale with diff size.

The footnote is the punchline of the lecture so far: AI makes it trivial to generate 900 lines, which makes small PRs *more* important, not less. Volume went up; review capacity did not.

“What / why / tested” is worth writing on the board — most students write only *what*, and *why* is the part reviewers actually need.

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

2026-07-22

AI-Assisted Software Development

- └ Prompting
 - └ Outline

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

Five prompt patterns for code

- ❶ **Give it what it can't see** — the surrounding code, the schema, the error.
- ❷ **Ask for one thing**, explicitly — one change per turn, then iterate.
- ❸ **Be specific about constraints** — version, libraries, style, performance, error semantics.
- ❹ **Ask for tests** — “add a pytest for the empty-input case.”
- ❺ **Ask for alternatives when designing** — “three storage options with trade-offs.”

2026-07-22

AI-Assisted Software Development

└ Prompting

└ Five prompt patterns for code

~10 min — make this interactive.

Point: pattern 1 does the heavy lifting. Most bad output is an *information* problem, not a model problem — the assistant couldn't see your schema, your versions, or the actual traceback.

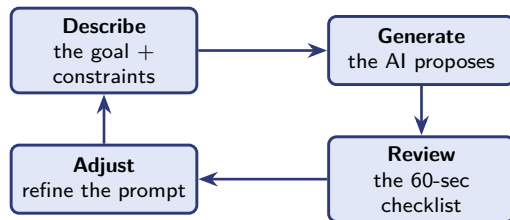
Do it live: take a vague request (“fix this function”), watch it guess, then re-ask with the error text and the surrounding code pasted in. The difference is stark and needs no commentary.

Pattern 2 (one thing per turn) is the discipline students resist most — they ask for a feature, tests and a refactor at once, then can't tell which part broke.

Pattern 5 is under-used and worth selling: asking for three options with trade-offs uses the model as a *thinking partner* rather than a code dispenser, and it's the one place its breadth genuinely beats yours.

- ❶ Give it what it can't see — the surrounding code, the schema, the error.
- ❷ Ask for one thing, explicitly — one change per turn, then iterate.
- ❸ Be specific about constraints — version, libraries, style, performance, error semantics.
- ❹ Ask for tests — “add a pytest for the empty-input case.”
- ❺ Ask for alternatives when designing — “three storage options with trade-offs.”

Prompting is a loop, not a one-shot



Each turn is small and reviewable. You stay the author — the assistant drafts; you decide.

2026-07-22

AI-Assisted Software Development

└ Prompting

└ Prompting is a loop, not a one-shot



Each turn is small and reviewable. You stay the author — the assistant drafts; you decide.

~6 min. Point: it is a loop. Beginners treat a bad first answer as a verdict on the tool and give up; experienced users expect to go round three times.

Note the loop is small on purpose — each turn stays reviewable. That’s what keeps the Review step to 60 seconds instead of an afternoon.

The last line is the ethical and professional core of the lecture: you stay the author. You sign the commit, you defend it in review, you get paged when it breaks. “The AI wrote it” is not a defence anywhere it matters.

Note Review feeds *Adjust the prompt*, not “fix it by hand.” Fixing by hand silently teaches you nothing about steering the tool.

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review**
- 5 When not

2026-07-22

AI-Assisted Software Development

└ Review

└ Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review**
- 5 When not

Four failure modes of AI-generated code

A — Hallucinated APIs

calls a function / argument that does not exist

B — Plausible but wrong

looks right; the logic is subtly off

C — Architectural drift

ignores the codebase's patterns

D — Silent over-confidence

“safely” swallows errors, hides failure

2026-07-22

AI-Assisted Software Development

└ Review

└ Four failure modes of AI-generated code

Four failure modes of AI-generated code



~10 min — the heart of the lecture.

Point: four *distinct* failure modes, in rising order of danger — and note that danger rises as detectability falls.

A is harmless: it crashes immediately, you fix it. **B** is the expensive one — it runs, returns plausible numbers, and is wrong; only tests or domain knowledge catch it. **C** compounds over months into a codebase with five conventions. **D** is the worst: except Exception: pass means the failure never surfaces at all.

Tie back to NB 27: the model predicts *plausible* continuations. Plausible-but-wrong isn't a bug in the assistant — it is the objective function, showing up in your codebase.

Ask the room: “Which is most dangerous?” Most say A (it's the most visible). The realisation that D is worse because it is invisible is the moment worth engineering.

The 60-second review checklist

Run this on every AI-generated change before you accept or merge

- ❶ **Does it run?** Even just the import line.
- ❷ **Are there tests?** If not, ask for them.
- ❸ **Does it use our libraries / patterns?** Grep for one similar function.
- ❹ **Does it swallow errors?** Search the diff for `except Exception, or {}, pass`.
- ❺ **Does it do exactly what was asked** — no more, no less?

60 seconds; catches $\approx 80\%$ of failures. The rest is what real testing and code review are for.

2026-07-22

AI-Assisted Software Development

└ Review

└ The 60-second review checklist

~**8 min.** Point: the practical takeaway of the deck — five questions, one minute, and each maps onto a failure mode from the previous slide.

Make the mapping explicit: Q1 catches **A**, Q2 catches **B**, Q3 catches **C**, Q4 catches **D**, and Q5 catches scope creep — the assistant helpfully refactoring things you never asked about.

Q4 is the one to demonstrate: literally search the diff for `except Exception, pass, or {}`. Ten seconds, and it catches the most dangerous category.

Be honest about the 80 % — this is a filter, not a guarantee. It is what stands between “I skimmed it” and “I reviewed it”; tests and real code review do the rest.

Tell them to photograph this slide. It’s the one they’ll actually reuse on Monday.

Run this on every AI-generated change before you accept or merge

- ❶ Does it run? Even just the import line.
- ❷ Are there tests? If not, ask for them.
- ❸ Does it use our libraries / patterns? Grep for one similar function.
- ❹ Does it swallow errors? Search the diff for `except Exception, or {}, pass`.
- ❺ Does it do exactly what was asked — no more, no less?

60 seconds; catches $\approx 80\%$ of failures. The rest is what real testing and code review are for.

Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

2026-07-22

AI-Assisted Software Development

└─ When not

└─ Outline

- 1 IDEs
- 2 Git
- 3 Prompting
- 4 Review
- 5 When not

When *not* to use the AI assistant

Mental model

AI assistants are most valuable when you *could* write the code yourself but would rather not. They are least valuable when you don't yet understand the problem.

- Skip it when the task is to *learn* the concept — write the first version yourself.
- Skip it for security- / correctness-critical logic you can't fully review.
- **Companion notebook:** hands-on Git, PR, and review exercises are in **Notebook 51** — recommended *early* so the workflow pays off everywhere.

2026-07-22

AI-Assisted Software Development

- └ When not
- └ When *not* to use the AI assistant

~8 min — close. Point: the mental model in the block is the sentence to leave them with. Say it slowly.

“**Skip it when the task is to learn**” deserves directness, because it's about *this course*: if they autocomplete their way through Module 1, they will not be able to read their own capstone. The struggle is the mechanism, not an obstacle to route around. Using the assistant to *explain* is fine; using it to *write* the exercise is not.

The security bullet is the professional one: never ship what you cannot review. Code you don't understand is code you cannot debug at 3 a.m.

Ask the room: “Where's the line for you personally?” Genuinely open — it varies by seniority, and hearing the disagreement is the point.

Send them to Notebook 51 early — the workflow compounds across every later module.

When not to use the AI assistant

Mental model

AI assistants are most valuable when you *could* write the code yourself but would rather not. They are least valuable when you don't yet understand the problem.

- Skip it when the task is to *learn* the concept — write the first version yourself.
- Skip it for security- / correctness-critical logic you can't fully review.
- **Companion notebook:** hands-on Git, PR, and review exercises are in **Notebook 51** — recommended *early* so the workflow pays off everywhere.